DS4001-25SP-HW1：搜索

黄思睿　PB23151782

2025 年 4 月 7 日

0　代码理解 [20%]

(a) **[截图]**

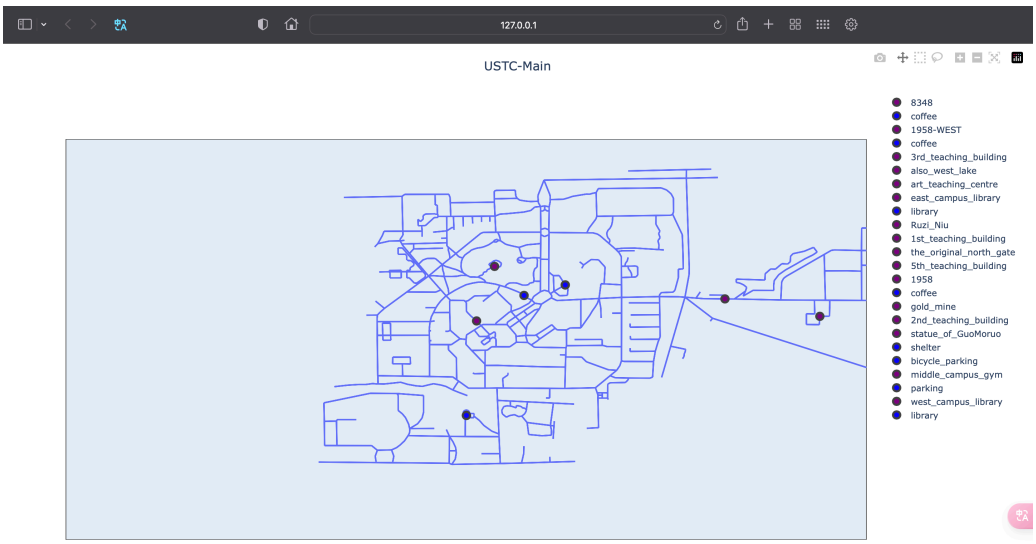

图 1: 初步运行

0.1　整体结构分析 [6%]

(b) **[配对]**

- (1):C
- (2):E
- (3):F
- (4):D
- (5):A
- (6):B
- (7):G

0.2 详细代码阅读 [10%=2%*5]

(c) **[多选]** A、B、E

(d) **[单选]** B

(e) **[多选]** A、B、C

(f) **[多选]** A、B、C、D、E

(g) **[简答]**

`verbose` 的作用

`verbose` 是一个 **调试输出控制变量**，用于控制算法执行过程中输出的详细程度。它的值是一个 **整数**（如 `0`，`1`，`2`，`3`），不同数值对应不同级别的输出信息：

具体行为分析

1. **当** `verbose >= 1`：
   - 输出 **关键结果**，例如：
     - 当搜索结束时打印 `numStatesExplored`、`pathCost` 和 `actions`。
     - 如果搜索空间完全探索但未找到解，打印 `Searched the entire search space!`。
2. **当** `verbose >= 2`：
   - 输出 **每一步探索的详细信息**，例如：
     - 每次探索一个状态时，打印当前状态和累计代价（如 `Exploring {state} with pastCost {pastCost}`）。
3. **当** `verbose >= 3`：
   - 输出 **最细粒度的调试信息**，例如：
     - 每次状态转移时，打印转移的来源、目标状态及代价（如 `{state} => {newState} (Cost: {pastCost} + {cost})`）。

设计目的

- **调试用途**：通过调整 `verbose` 的值，开发者可以在不修改代码的情况下，控制是否输出日志以及日志的详细程度。
- **灵活性**：
  - 在开发阶段，可设置 `verbose=3` 以追踪算法的每一步操作。
  - 在最终运行或性能敏感场景中，可设置 `verbose=0` 关闭所有输出，仅保留核心逻辑。

图 2: 0.2(g) deepseek 简答

1 问题 1：查找最短路径 [29%]

1.1 建模 [10%=6%+4%]

(a) **[代码]**

```python
class ShortestPathProblem(SearchProblem):
    """
    Defines a search problem that corresponds to finding the shortest path
    from `startLocation` to any location with the specified `endTag`.
    """

    def __init__(self, startLocation: str, endTag: str, cityMap: CityMap):
        self.startLocation = startLocation
        self.endTag = endTag
        self.cityMap = cityMap
```

```python

    def startState(self) -> State:
        # BEGIN_YOUR_CODE (our solution is 1 line of code, but don't worry if you deviate from this)
        return State(self.startLocation, None)  # 初始状态为起点
        # END_YOUR_CODE

    def isEnd(self, state: State) -> bool:
        # BEGIN_YOUR_CODE (our solution is 1 line of code, but don't worry if you deviate from this)
        return self.endTag in self.cityMap.tags[state.location]  # 检查当前节点是否为目标标签
        # END_YOUR_CODE

    def successorsAndCosts(self, state: State) -> List[Tuple[str, State, float]]:
        # BEGIN_YOUR_CODE (our solution is 4 lines of code, but don't worry if you deviate from this)
        successors = []
        current = state.location
        for neighbor, cost in self.cityMap.distances[current].items():
            # 将动作名称设为邻居的位置标签
            successors.append((neighbor, State(neighbor, None), cost))
        return successors
        # END_YOUR_CODE
```

(b) **[代码]**

```python
    def getUSTCShortestPathProblem() -> ShortestPathProblem:
    """
    Create your own search problem using the map of USTC, specifying your own
    `startLocation`/`endTag`.

    Run `python mapUtil.py > readableUSTCMap.txt` to dump a file with a list of
    locations and associated tags; you might find it useful to search for the following
    tag keys (amongst others):
        - `landmark=` - Hand-defined landmarks (from `data/USTC-landmarks.json`)
        - `amenity=`  - Various amenity types (e.g., "coffee", "food")
    """
    cityMap = createUSTCMap()

    # BEGIN_YOUR_CODE (our solution is 2 lines of code, but don't worry if you deviate from this)
    startLocation = locationFromTag(makeTag("landmark", "8348"), cityMap)  # 西区8348
    endTag = makeTag("landmark", "3rd_teaching_building")  # 三教标签
     # END_YOUR_CODE
    return ShortestPathProblem(startLocation, endTag, cityMap)
```

[截图]

图 3: 从 8348 到西区三教路径可视化

1.2 算法 [19%=6%+5%+2%+6%]

(c) **[简答] 归纳基础:**

$$d_s^{(0)}(u) = \begin{cases} 0, & u = s \\ +\infty, & \text{其他情况} \end{cases}$$

此时:

$$d_s^{(1)}(u) = \min\{d_s^{(0)}(u), d_s^{(0)}(v_0) + w_{v_0 u}\}, \quad \text{其中} v_0 = s.$$

因此:

$$d_s^{(1)}(u) = \min\{+\infty, 0 + w_{su}\} = w_{su}.$$

由于实际最短路径满足 $d_s(u) \le w_{su}$，故:

$$d_s(u) \le d_s^{(1)}(u) \le d_s^{(0)}(u).$$

归纳假设: 假设对 $k = m$，有:

$$d_s(u) \le d_s^{(m+1)}(u) \le d_s^{(m)}(u).$$

归纳步骤: 当 $k = m + 1$ 时,

$$d_s^{(m+2)}(u) = \min\left\{d_s^{(m+1)}(u), \min_{v \in \{v_0, \ldots, v_{m+1}\}} (d_s^{(m+1)}(v) + w_{vu})\right\}.$$

下界： 由归纳假设，

$$d_s^{(m+1)}(u) \geq d_s(u), \quad 且 d_s^{(m+1)}(v_{m+1}) \geq d_s(v_{m+1}).$$

根据三角不等式：

$$d_s(u) \leq d_s(v_{m+1}) + w_{v_{m+1}u} \leq d_s^{(m+1)}(v_{m+1}) + w_{v_{m+1}u}.$$

因此：

$$d_s^{(m+2)}(u) \geq d_s(u).$$

上界： $d_s^{(m+2)}(u)$ 是 $d_s^{(m+1)}(u)$ 和新路径中的较小值，故：

$$d_s^{(m+2)}(u) \leq d_s^{(m+1)}(u).$$

综上，对所有 k 和 u，有：

$$d_s(u) \leq d_s^{(k+1)}(u) \leq d_s^{(k)}(u).$$

(d) **[简答]**

1.2 (d) 证明 $d_s^{(k)}(v_k) = d_s(v_k)$

归纳法：

基础情况： 当 $k = 0$ 时，$v_0 = s$，

$$d_s^{(0)}(s) = 0 = d_s(s).$$

归纳假设： 假设对所有 $k' < k$，有：

$$d_s^{(k')}(v_{k'}) = d_s(v_{k'}).$$

归纳步骤： 在第 k 次迭代中，v_k 是未处理顶点中 $d_s^{(k)}(v)$ 最小的。假设存在更短路径 $d_s(v_k) < d_s^{(k)}(v_k)$，则该路径必经过未处理顶点 u。

由归纳假设，已处理顶点的 d 值正确，且：

$$d_s^{(k)}(u) \geq d_s^{(k)}(v_k) \quad (因 v_k \text{ 是当前最小值}).$$

因此，路径长度：

$$d_s(u) + w_{uv_k} \geq d_s^{(k)}(v_k),$$

与假设矛盾。

故：

$$d_s^{(k)}(v_k) = d_s(v_k).$$

(e) **[判断]** 不能。

(f) **[简答]**

（1）理由：Dijkstra 算法的正确性依赖于“贪心选择”——一旦某个节点被标记为“已处理”，其到起点的最短距离就被永久确定。这一性质成立的前提是图中所有边权非负。若存在负边权，可能导致以下问题：

i). 负权边的干扰：假设节点 A 已被处理，之后发现一条从 A 到 B 的负权边，使得 B 的距离更新后更小。但由于 Dijkstra 算法不会重新处理已处理的节点，B 的正确最短距离将被遗漏。负权环的无解性：

ii). 若图中存在负权环，最短路径可能趋向负无穷，此时问题本身无解。

（2）进行如下循环

i). 初始化所有顶点的距离，$d_s(s) = 0$，其他顶点为 ∞。

ii). 对于每条边 (u, v, w)，进行 $|V| - 1$ 轮“操作”，使得 $d_s(v) = \min(d_s(v), d_s(u) + w)$。

iii). 额外进行一轮“操作”检查负权环（如果某条边仍然能被“操作”，则图中存在负权环）。

2 问题 2：查找带无序途径点的最短路径 [17%]

2.1 建模 [10%=6%+4%]

(a) **[代码]**

```python
class WaypointsShortestPathProblem(SearchProblem):
    """
    Defines a search problem that corresponds to finding the shortest path from
    `startLocation` to any location with the specified `endTag` such that the path also
    traverses locations that cover the set of tags in `waypointTags`.

    Think carefully about what `memory` representation your States should have!
    """
    def __init__(
        self, startLocation: str, waypointTags: List[str], endTag: str, cityMap: CityMap
    ):
        self.startLocation = startLocation
        self.endTag = endTag
        self.cityMap = cityMap

        # We want waypointTags to be consistent/canonical (sorted) and hashable (tuple)
        self.waypointTags = tuple(sorted(waypointTags))

    def startState(self) -> State:
        # BEGIN_YOUR_CODE (our solution is 1 line of code, but don't worry if you deviate from this)
        return State(self.startLocation,memory=tuple(1 if tag in self.cityMap.tags[self.startLocation]
                                                     else 0 for tag in self.waypointTags))
        # END_YOUR_CODE

    def isEnd(self, state: State) -> bool:
        # BEGIN_YOUR_CODE (our solution is 1 lines of code, but don't worry if you deviate from this)
        return self.endTag in self.cityMap.tags[state.location] and all(state.memory)
        # END_YOUR_CODE

    def successorsAndCosts(self, state: State) -> List[Tuple[str, State, float]]:
```

```python
        # BEGIN_YOUR_CODE (our solution is 13 lines of code, but don't worry if you deviate from this)
        neighbors = []
        for neighbor, distance in self.cityMap.distances[state.location].items():
            state_memory = tuple(max(state.memory[i],
                                     1 if tag in self.cityMap.tags[neighbor] else 0)
                                 for i, tag in enumerate(self.waypointTags))
            neighbors.append((neighbor, State(neighbor, memory=tuple(state_memory),), distance))
        return neighbors
        # END_YOUR_CODE
```

(b) **[代码]**

```python
    def getUSTCWaypointsShortestPathProblem() -> WaypointsShortestPathProblem:
    """
    Create your own search problem using the map of USTC, specifying your own
    `startLocation`/`waypointTags`/`endTag`.

    Similar to Problem 1b, use `readableUSTCMap.txt` to identify potential
    locations and tags.
    """
    cityMap = createUSTCMap()
    # BEGIN_YOUR_CODE (our solution is 3 lines of code, but don't worry if you deviate from this)
    startLocation = locationFromTag(makeTag("landmark", "8348"), cityMap)
    waypointTags = [makeTag("landmark", "1958-WEST"), makeTag("landmark", "west_campus_library")]
    endTag = makeTag("landmark", "3rd_teaching_building")
    # END_YOUR_CODE
    return WaypointsShortestPathProblem(startLocation, waypointTags, endTag, cityMap)
```

[截图]

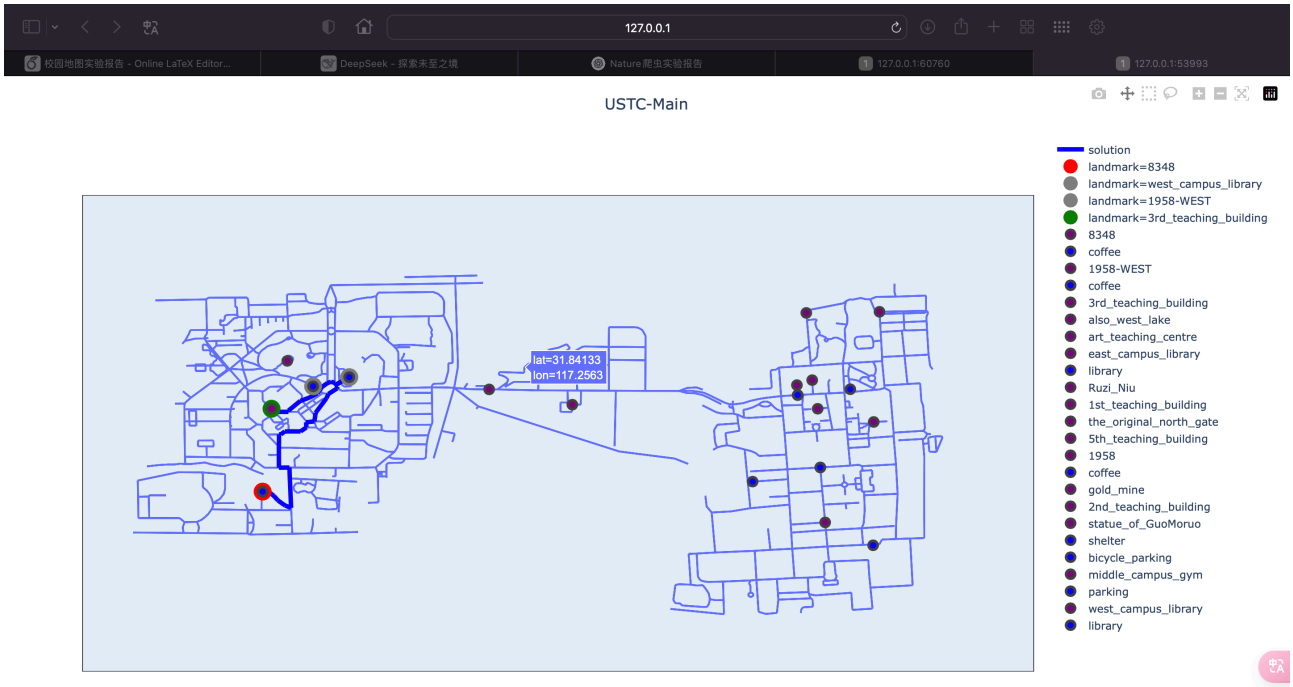

图 4: 从 8348 到西区三教，途径西图和西区 1958 路径可视化

2.2 算法 [7%=2%+5%]

(c) **[判断]** 不会

(d) **[简答]**

小李的质疑不成立。

反驳理由：

状态设计：WaypointsShortestPathProblem 的状态包含已访问的途径点集合(存储于 `State.memory`)。即使路径到达终点 B，若未完成所有途径点，状态仍未满足终止条件，算法会继续探索。

Dijkstra 算法的性质：

- Dijkstra 算法中，$d_s^{(k)}(u)$ 是当前已知的从起点到 u 的最短路径估计值。
- 当路径从 B 返回访问 X 时，新状态（$B \to X \to B$）的总代价可能大于之前路径的代价。由于 Dijkstra 算法优先处理代价更小的路径，正确的最短路径（完成所有途径点）会被优先探索。

3 问题 3：使用 A* 加快搜索速度 [32%]

3.1 将 UCS 转化为 A*[4%]

(a) **[代码]**

```python
def aStarReduction(problem: SearchProblem, heuristic: Heuristic) -> SearchProblem:
    class NewSearchProblem(SearchProblem):
        def __init__(self):
            # BEGIN_YOUR_CODE (our solution is 3 line of code, but don't worry if you deviate from this)
            self.problem = problem
            self.heuristic = heuristic
            self.startLocation = problem.startLocation  # 初始化 startLocation 属性
            self.endTag = problem.endTag  # 初始化 endTag 属性
            self.cityMap = problem.cityMap  # 初始化 cityMap 属性
            # END_YOUR_CODE

        def startState(self) -> State:
            # BEGIN_YOUR_CODE (our solution is 1 line of code, but don't worry if you deviate from this)
            return self.problem.startState()
            # END_YOUR_CODE

        def isEnd(self, state: State) -> bool:
            # BEGIN_YOUR_CODE (our solution is 1 line of code, but don't worry if you deviate from this)
            # 新问题的结束条件与原问题相同
            return self.problem.isEnd(state)
            # END_YOUR_CODE

        def successorsAndCosts(self, state: State) -> List[Tuple[str, State, float]]:
            # BEGIN_YOUR_CODE (our solution is 7 lines of code, but don't worry if you deviate from this)
            successors = self.problem.successorsAndCosts(state)
            new_successors = []
            for action, next_state, cost in successors:
                # 计算新的代价，结合启发式函数
                new_cost = cost + self.heuristic.evaluate(next_state) - self.heuristic.evaluate(state)
                new_successors.append((action, next_state, new_cost))
            return new_successors
            # END_YOUR_CODE

    return NewSearchProblem()
```

3.2 实现启发式函数 [18%=3%+6%+3%+6%]

(b) **[简答]**

对于一个启发式函数 $h(n)$，如果对于图中任意的节点 n 以及它的任意后继节点 n'，都满足：

$$h(n) \leq c(n, a, n') + h(n')$$

其中 $c(n, a, n')$ 是从节点 n 通过动作 a 到达节点 n' 的代价，那么称该启发式函数 h 是一致的。

直线距离启发式函数：

直线距离启发式函数 $h_{SLD}(n)$ 定义为节点 n 到目标节点的直线距离。

证明过程：

设 n 是图中的一个节点，n' 是 n 的一个后继节点，$c(n, a, n')$ 是从 n 到 n' 的实际路径代价，$h_{SLD}(n)$ 是 n 到目标节点 g 的直线距离，$h_{SLD}(n')$ 是 n' 到目标节点 g 的直线距离。

根据三角形不等式，在一个三角形中，任意两边之和大于第三边。考虑由节点 n、n' 和目标节点 g 构成的三角形，从 n 到 g 的直线距离一定小于或等于从 n 到 n' 的实际路径代价加上从 n' 到 g 的直

线距离，即：

$$h_{SLD}(n) \leq c(n, a, n') + h_{SLD}(n')$$

因此，直线距离启发式函数 h_{SLD} 是一致的，这保证了 A* 算法的正确性。

(c) **[代码]**

```python
class StraightLineHeuristic(Heuristic):
    """
    Estimate the cost between locations as the straight-line distance.
        > Hint: you might consider using `computeDistance` defined in `mapUtil.py`
    """
    def __init__(self, endTag: str, cityMap: CityMap):
        self.endTag = endTag
        self.cityMap = cityMap

        # Precompute
        # BEGIN_YOUR_CODE (our solution is 4 lines of code, but don't worry if you deviate from this)
        # 预计算所有具有目标标签的位置
        self.end_locations = [loc for loc, tags in self.cityMap.tags.items() if endTag in tags]
        # END_YOUR_CODE

    def evaluate(self, state: State) -> float:
        # BEGIN_YOUR_CODE (our solution is 6 lines of code, but don't worry if you deviate from this)
        current_location = state.location
        min_distance = float('inf')
        # 计算当前位置到所有目标位置的直线距离，并取最小值
        for end_location in self.end_locations:
            if current_location in self.cityMap.geoLocations and end_location in self.cityMap.geoLocations:
                distance = computeDistance(self.cityMap.geoLocations[current_location], self.cityMap.geoLocations[
                min_distance = min(min_distance, distance)
        return min_distance
        # END_YOUR_CODE
```

对于问题 2，我们使用不带途径点的最短路径长度作为启发式函数。

(d) **[简答]**

不带途径点的最短路径长度启发式函数：

设 $h_{SP}(n)$ 表示节点 n 到目标节点的不带途径点的最短路径长度。

证明过程：

同样依据一致性的定义，对于任意节点 n 及其后继节点 n'，我们需要证明：

$$h_{SP}(n) \leq c(n, a, n') + h_{SP}(n')$$

假设 $h_{SP}(n)$ 是节点 n 到目标节点 g 的最短路径长度（不考虑途径点），$h_{SP}(n')$ 是节点 n' 到 g 的最短路径长度，$c(n, a, n')$ 是从 n 到 n' 的实际路径代价。

从 n 到 g 的最短路径（不带途径点）有两种可能：

- 直接路径：$n \to g$，此时 $h_{SP}(n) = c(n, a, g)$。
- 间接路径：$n \to n' \to g$，此时路径长度为 $c(n, a, n') + h_{SP}(n')$。

由于 $h_{SP}(n)$ 是 n 到 g 的最短路径，因此必然满足：

$$h_{SP}(n) \leq c(n, a, n') + h_{SP}(n')$$

因此，不带途径点的最短路径长度启发式函数 h_{SP} 是一致的，保证了 A* 算法的正确性。

(e) **[代码]**

```python
class NoWaypointsHeuristic(Heuristic):
    """
    Returns the minimum distance from `startLocation` to any location with `endTag`,
    ignoring all waypoints.
    """
    def __init__(self, endTag: str, cityMap: CityMap):
        # Precompute
        # BEGIN_YOUR_CODE (our solution is 14 lines of code, but don't worry if you deviate from this)
        self.shortestPaths = {}

        targetLocations = sorted(
            [location for location, tags in cityMap.tags.items() if endTag in tags]
        )

        for target in targetLocations:
            pathProblem = ShortestPathProblem(target, "NoGoal", cityMap)
            pathFinder = UniformCostSearch(verbose=0)
            pathFinder.solve(pathProblem)
            for state, cost in pathFinder.pastCosts.items():
                loc = state.location
                if loc not in self.shortestPaths or self.shortestPaths[loc] > cost:
                    self.shortestPaths[loc] = cost
        # END_YOUR_CODE

    def evaluate(self, state: State) -> float:
        # BEGIN_YOUR_CODE (our solution is 1 line of code, but don't worry if you deviate from this)
        return self.shortestPaths[state.location]
        # END_YOUR_CODE
```

3.3 利用合肥市地图对比运行时间 [10%=4%+6%]

(f) **[代码]**

```python
def getHefeiShortestPathProblem(cityMap: CityMap) -> ShortestPathProblem:
    """
    Create a search problem using the map of Hefei
    """
    startLocation=locationFromTag(makeTag("landmark", "USTC"), cityMap)
    endTag=makeTag("landmark", "Chaohu")
    # BEGIN_YOUR_CODE (our solution is 1 lines of code, but don't worry if you deviate from this)
    return ShortestPathProblem(startLocation, endTag, cityMap)
    # END_YOUR_CODE

def getHefeiShortestPathProblem_withHeuristic(cityMap: CityMap) -> ShortestPathProblem:
    """
    Create a search problem with Heuristic using the map of Hefei
    """
    startLocation=locationFromTag(makeTag("landmark", "USTC"), cityMap)
    endTag=makeTag("landmark", "Chaohu")
```

```python
    # BEGIN_YOUR_CODE (our solution is 2 lines of code, but don't worry if you deviate from this)
    # 创建直线距离启发式函数
    heuristic = StraightLineHeuristic(endTag, cityMap)
    '''
    可替换成
    heuristic = NoWaypointsHeuristic(endTag, cityMap)
    用忽视中间节点算法
    '''
    # 创建最短路径问题
    problem = ShortestPathProblem(startLocation, endTag, cityMap)
    # 使用 A* 到 UCS 的归约
    new_problem = aStarReduction(problem, heuristic)
    return new_problem
    # END_YOUR_CODE
```

[时间对比]

```
(lab1) (base) halo@HalodeMacBook-Air Project % python grader.py 3f-with_Heuristic
========== START GRADING
----- START PART 3f-with_Heuristic: shortest path through Hefei
Total distance: 73155.31956705352
----- END PART 3f-with_Heuristic [took 0:00:01.350416 (max allowed 50 seconds), 1/1 points]

Note that the hidden test cases do not check for correctness.
They are provided for you to verify that the functions do not crash and run within the time limit.
Points for these parts not assigned by the grader (indicated by "--").
========== END GRADING [1/1 points + 0/0 extra credit]
```

图 5: with Heuristic （用 StraightLineHeuristic 方法实现）运行时间

```
(lab1) halo@HalodeMacBook-Air Project % python grader.py 3f-with_Heuristic
========== START GRADING
----- START PART 3f-with_Heuristic: shortest path through Hefei
Total distance: 73155.31956705352
----- END PART 3f-with_Heuristic [took 0:00:02.479586 (max allowed 50 seconds), 1/1 points]

Note that the hidden test cases do not check for correctness.
They are provided for you to verify that the functions do not crash and run within the time limit.
Points for these parts not assigned by the grader (indicated by "--").
```

图 6: with Heuristic （用 NoWaypointsHeuristic 方法实现）运行时间

```
(lab1) halo@HalodeMacBook-Air Project % python grader.py 3f-without_Heuristic
========== START GRADING
----- START PART 3f-without_Heuristic: shortest path through Hefei
Total distance: 73155.31956705352
----- END PART 3f-without_Heuristic [took 0:00:02.571700 (max allowed 50 seconds), 1/1 points]

Note that the hidden test cases do not check for correctness.
They are provided for you to verify that the functions do not crash and run within the time limit.
Points for these parts not assigned by the grader (indicated by "--").
========== END GRADING [1/1 points + 0/0 extra credit]
```

图 7: without Heuristic 运行时间

可以看到，with Heuristic 中使用 StraightLineHeuristic 方法运行时间约为 1.350416s，使用 NoWaypointsHeuristic 方法的运行时间约为 2.469586s (这可能与 NoWaypointsHeuristic 算法一开始的预计算开销更大有关)

without Heuristic 的运行时间约为 2.571700s

(g) [**简答**] **现有缺陷：**

- **状态空间爆炸：**当地图扩大（如合肥市）时，状态数量急剧增加，导致搜索时间变长。
- **启发式函数局限性：**当前启发式函数（如直线距离）可能无法有效剪枝，尤其是在复杂城市道路中。
- **实时性不足：**对于大规模地图，实时路径规划可能无法满足用户需求。

可能的解决方案：

- **分层搜索（Hierarchical Search）：**
  - **思路：**将地图划分为多个层次（如主干道、次干道、支路），先在高层地图上进行粗略搜索，再在低层地图上精细化路径。
  - **步骤：**
    (a) 预处理生成高层地图（如仅保留主干道）。
    (b) 在高层地图上使用 A* 算法找到大致路径。
    (c) 在低层地图上细化路径，确保符合具体道路限制。
  - **优势：**减少搜索空间，提高效率。
- **动态权重启发式函数：**
  - **思路：**结合实时交通数据（如拥堵情况）动态调整启发式函数的权重，优先选择更高效的路径。
  - **实现：**
    (a) 在 $h(n)$ 中加入实时权重（如 $h(n) =$ 直线距离 $\times$ 拥堵系数）。
    (b) 拥堵系数根据历史数据或实时传感器信息更新。
- **预处理最短路径表：**
  - **思路：**预先计算所有关键点（如地标、交叉口）之间的最短路径，查询时直接组合这些路径。
  - **步骤：**
    (a) 使用 Floyd-Warshall 或 Dijkstra 算法预处理关键点间的最短路径。
    (b) 对于用户查询，将路径分解为关键点序列，组合预处理结果。
  - **优势：**将复杂问题分解为子问题，减少实时计算量。

体验反馈 [2%]

(a) [**必做**] 五个半天（?）

(b) [**选做**] 你的答案